4FT901CC24_CSE

Research Methodology and Seminar

B.Tech. Semester – VI(AY:2025-26)

Report on

Failure Recovery System Using Agentic AI in Software Development

Report Prepared By:

Meet Patel
23BCE177
Vedant Mehta
23BCE180

Guide: Mr. AjayKumar Patel

School of Technology,
Institute of Technology,
Nirma University ,
Ahmedabad-382481

Table of Contents:

Contents

Table of Contents: 2
Abstract 4
1.1 Background 6
1.2 Motivation 6
1.3 Research Gap 6
1.4 Objectives 7
1.5 Contributions of the Report 8
2. Background and Foundations 9
2.1 Software Failure Types 9
2.2 Automated Program Repair (APR) 9
2.3 Machine Learning in Program Repair 10
2.4 Large Language Models in Software Engineering 10
2.5 Agentic AI Systems 11
2.6 Self-Healing Software Systems 12
3. Problem Statement 13
3.1 Overview of the Problem 13
3.2 Formal Problem Definition 13
3.3 Challenges in Failure Recovery 14
3.4 Research Question 14
3.5 Goal of the Proposed System 14
4. Proposed Failure Recovery System 15
4.1 System Overview 15
4.2 Multi-Agent Architecture 15
4.2.1 Failure Detection Agent 15
4.2.2 Code Localization Agent 16
4.2.3 Debugging Agent 16
4.2.4 Patch Generation Agent 17
4.2.5 Patch Application Agent 18
4.2.6 Validation Agent 18

4.2.7 Coordinator Agent 19
4.3 Recovery Workflow 20
4.4 Tool Integration 20
5. Methodology 21
5.1 Dataset Description 21
5.2 Proposed Experimental Setup 21
5.3 Model Configuration 22
5.4 Evaluation Metrics 22
5.4.1 Repair Performance Metrics 22
5.4.2 Recovery Efficiency Metrics 23
5.4.3 Reliability and Safety Metrics 23
5.5 Comparative Baseline 23
6.1 Classical Automated Program Repair 24
6.2 Machine Learning-Based Repair 24
6.3 Large Language Model-Based Repair 25
6.4 Agentic AI-Based Recovery Systems 25
6.5 Comparative Analysis of Existing Approaches 26
6.6 Identified Research Gap 26
7. Conclusion 27
8.References 28

Abstract

Modern software systems frequently experience failures such as compilation errors, runtime exceptions, logical defects, and test case failures, particularly in continuous integration and deployment (CI/CD) environments. As software systems grow in size and complexity, manual debugging and failure recovery become increasingly time-consuming and error-prone. Although traditional automated program repair techniques and recent Large Language Model (LLM)-based approaches have shown potential in automated debugging, they often lack structured reasoning, iterative validation, and end-to-end recovery workflows suitable for real-world repositories.

This report proposes a Failure Recovery System using Agentic AI, designed as a structured multi-agent framework for automated software debugging and recovery. The system consists of specialized agents responsible for failure detection, code localization, debugging analysis, patch generation, patch application, validation, and coordination. These agents operate collaboratively within an iterative recovery loop that detects failures, generates corrective patches, applies fixes, and validates outcomes using automated test cases until recovery is achieved or a predefined retry limit is reached.

The proposed framework focuses on architectural design and methodological analysis rather than full-scale implementation. Evaluation is discussed using benchmark datasets such as Defects4J and recovery-oriented metrics including repair success rate and Mean Time to Recovery (MTTR). The system aims to improve repair robustness, reduce recovery time, and contribute

toward the development of self-healing software systems for modern development environments.

1. Introduction

1.1 Background

Software development has become more complex due to large codebases, distributed systems, and continuous integration and deployment (CI/CD) practices. Modern software projects frequently encounter failures such as compilation errors, runtime exceptions, logical bugs, dependency conflicts, and failed test cases. These failures can interrupt development workflows and delay product delivery.

In CI/CD environments, automated testing and integration pipelines run continuously. Even a small defect in code can cause the entire pipeline to fail. Therefore, fast and efficient failure recovery has become an important requirement in software engineering.

1.2 Motivation

Currently, most debugging and failure recovery tasks are performed manually by developers. This process involves analyzing error logs, identifying the faulty code, writing corrections, and re-running tests. Manual debugging requires time, expertise, and repeated effort. In large projects, this can significantly increase development cost and recovery time.

Traditional Automated Program Repair (APR) systems attempt to automate bug fixing using search-based or template-based approaches. However, these systems often lack intelligent reasoning and adaptability. With the emergence of Large Language Models (LLMs) capable of understanding and generating code, there is an opportunity to design intelligent systems that can automatically analyze and recover from failures in a structured way.

This motivates the development of a failure recovery system that can operate autonomously using reasoning-based AI agents.

1.3 Research Gap

Although traditional repair systems and LLM-based tools exist, several limitations remain:

- Traditional APR systems rely on fixed repair templates or limited search strategies.
- Single-step LLM repair approaches lack structured planning and iterative validation.
- Existing systems mainly focus on patch generation rather than complete recovery workflows.
- Limited research has explored structured multi-agent collaboration for automated failure recovery in CI/CD environments.

There is a need for a unified framework that integrates detection, reasoning, patch generation, validation, and coordination in a structured and automated manner.

1.4 Objectives

The primary objectives of this project are:

1. To design a multi-agent failure recovery architecture using Agentic AI.
2. To automate detection, debugging, and validation of software failures.
3. To reduce Mean Time to Recovery (MTTR) in CI/CD workflows.
4. To evaluate the proposed system using benchmark datasets such as Defects4J.
5. To compare the proposed approach with traditional and single-agent repair methods.

1.5 Contributions of the Report

This report makes the following contributions:

- Proposes a structured multi-agent architecture for automated failure recovery.
- Integrates reasoning-based LLM agents with tool execution and validation loops.
- Introduces recovery-oriented evaluation metrics such as MTTR and repair attempts.
- Demonstrates the applicability of the approach on real-world bug datasets.
- Provides a comparative framework against existing repair methodologies.

2. Background and Foundations

2.1 Software Failure Types

Software failures occur when a program does not behave as expected. In modern development environments, failures can appear in different forms:

- **Compilation Errors** – Errors that prevent the program from building successfully.
- **Runtime Exceptions** – Errors that occur while the program is running, such as NullPointerException or IndexOutOfBoundsException.
- **Logical Bugs** – The program runs but produces incorrect output.
- **Test Case Failures** – Automated tests detect unexpected behavior.
- **CI/CD Pipeline Failures** – Build or integration processes stop due to code issues.

In large systems, these failures can delay releases and affect reliability. Therefore, an automated recovery mechanism is important for reducing downtime.

2.2 Automated Program Repair (APR)

Automated Program Repair (APR) refers to techniques that attempt to fix software bugs automatically without human intervention.

Early approaches such as Genetic Programming-based repair generated patches by modifying code randomly and checking if tests pass. Later, template-based methods applied predefined repair patterns to common bug types. Some systems introduced probabilistic models to rank possible patches based on learned patterns from human-written fixes.

Although APR has achieved partial success, it has several limitations:

- Limited reasoning ability
- Large search space

- Risk of generating incorrect but test-passing patches
- Difficulty handling large repositories

These limitations motivate more intelligent repair approaches.

2.3 Machine Learning in Program Repair

Machine learning techniques were introduced to improve patch generation. Neural networks were trained to learn patterns from existing bug fixes. Deep learning models attempted to predict corrected code based on context.

However, many machine learning-based systems:

- Focus only on small code snippets
- Struggle with large-scale repositories
- Lack iterative validation loops

While they improve prediction ability, they often lack structured decision-making mechanisms.

2.4 Large Language Models in Software Engineering

Large Language Models (LLMs) such as transformer-based models can understand and generate code. These models are trained on large datasets containing programming languages and natural language.

LLMs can:

- Generate code from instructions
- Explain bugs
- Suggest fixes
- Analyze error logs

Despite these advantages, single-step LLM repair systems may:

- Produce incorrect or incomplete patches

- Miss contextual information
- Fail to validate fixes properly

Therefore, LLMs require structured orchestration to work effectively in real-world debugging.

2.5 Agentic AI Systems

Agentic AI refers to AI systems that can:

- Reason about problems
- Plan actions
- Use tools
- Execute tasks
- Evaluate outcomes
- Adapt through iterative feedback

Unlike simple predictive models, agentic systems operate in loops. They observe the environment, take action, and adjust based on results.

In software debugging, agentic AI can:

- Read logs
- Analyze source code
- Generate patches
- Run tests
- Retry if needed

This structured workflow improves robustness.

2.6 Self-Healing Software Systems

Self-healing systems are systems that can automatically detect and recover from failures without human intervention.

The concept originates from autonomic computing models, where systems follow a control loop:

- Monitor
- Analyze
- Plan
- Execute

Your proposed system follows a similar structure but integrates reasoning-based AI agents within this loop.

By combining agentic AI and self-healing principles, the system moves toward automated and intelligent failure recovery.

3. Problem Statement

3.1 Overview of the Problem

Modern software development environments frequently experience failures during coding, testing, and integration phases. These failures may arise due to syntax errors, incorrect logic, missing dependencies, runtime exceptions, or changes in software components. In continuous integration and deployment (CI/CD) workflows, even a minor defect can interrupt automated testing pipelines and delay software releases.

Currently, recovery from such failures largely depends on manual debugging. Developers must examine error logs, locate the faulty code, implement corrections, and re-run tests repeatedly. This manual process is time-consuming, requires expertise, and increases development cost. Although Automated Program Repair (APR) systems exist, they often lack structured reasoning and do not provide a complete end-to-end recovery mechanism.

3.2 Formal Problem Definition

Let:

- **R** represent a software repository.
- **T** represent the associated test suite.
- **L** represent the failure logs generated after test execution.

Given a buggy version of repository **R**, when the test suite **T** is executed, failure logs **L** are produced indicating errors in the system.

The problem is to design an autonomous system **A** that can:

1. Detect failures from logs **L**.
2. Identify the faulty code region in **R**.
3. Generate a corrective patch.

4. Apply the patch to the repository.
5. Validate correctness by re-executing **T**.
6. Minimize the time required for successful recovery.

3.3 Challenges in Failure Recovery

Several challenges exist in automating failure recovery:

- Large codebases make fault localization difficult.
- Multiple possible fixes may exist for a single failure.
- Passing tests do not always guarantee semantic correctness.
- LLM-generated patches may contain formatting or logical errors.
- Iterative retries increase computational cost.

These challenges require a structured and intelligent system rather than a single-step repair approach.

3.4 Research Question

This work aims to address the following research question:

Can a structured multi-agent agentic AI system improve automated failure recovery performance and reduce recovery time compared to traditional and single-agent repair approaches?

3.5 Goal of the Proposed System

The primary goal is to develop a multi-agent failure recovery framework that integrates reasoning, planning, tool usage, and validation to achieve efficient and reliable automated debugging.

4. Proposed Failure Recovery System

4.1 System Overview

The proposed Failure Recovery System is a structured multi-agent framework designed to automatically detect, analyze, and fix software failures. The system operates in an iterative recovery loop, similar to a CI/CD pipeline. When a failure occurs during test execution, the system collects error logs, analyzes the issue, generates corrective patches, applies modifications, and validates the result using automated tests.

Unlike traditional repair systems that perform a single-step patch generation, the proposed system follows a coordinated workflow involving multiple specialized agents. Each agent performs a specific role, and the overall process continues until the failure is resolved or a predefined retry limit is reached.

The system architecture is modular, meaning each agent can operate independently but shares information through a central coordinator.

4.2 Multi-Agent Architecture

The system consists of the following agents:

4.2.1 Failure Detection Agent

Purpose: Identify the presence and type of failure.

Input:

- Test execution output
- Error logs

Process:

- Detect failing test cases
- Extract error type (e.g., AssertionError, NullPointerException)

- Capture stack trace information

Output:

- Structured failure summary including test name, error message, and error location

This output is passed to the Code Localization Agent.

4.2.2 Code Localization Agent

Purpose: Identify the most likely faulty source file and method.

Input:

- Failure summary
- Stack trace details
- Repository structure

Process:

- Map stack trace to source file
- Retrieve relevant code snippet
- Narrow down suspected method

Output:

- Suspected file path
- Extracted code snippet

This information is passed to the Debugging Agent.

4.2.3 Debugging Agent

Purpose: Analyze the failure and generate a repair hypothesis.

Input:

- Failure summary
- Extracted code snippet

Process:

- Use reasoning-based language model
- Analyze error context
- Identify potential bug cause
- Propose correction strategy

Output:

- Suggested patch logic
- Explanation of fix

This suggestion is sent to the Patch Generation Agent.

4.2.4 Patch Generation Agent

Purpose: Produce corrected code based on debugging analysis.

Input:

- Repair hypothesis
- Original code snippet

Process:

- Generate updated method or code block
- Ensure syntax correctness
- Maintain code structure

Output:

- Modified source code patch

The patch is passed to the Patch Application Agent.

4.2.5 Patch Application Agent

Purpose: Apply generated patch to repository.

Input:

- Generated patch
- Target file location

Process:

- Replace faulty code
- Save modified source file

Output:

- Updated repository version

The modified code is sent to the Validation Agent.

4.2.6 Validation Agent

Purpose: Verify correctness of applied patch.

Input:

- Updated repository

Process:

- Execute test suite
- Check if all tests pass
- Detect new compilation errors

Output:

- Success or failure result
- Updated logs if failure persists

If failure remains, information is returned to Coordinator Agent.

4.2.7 Coordinator Agent

Purpose: Manage workflow and decision logic.

Input:

- Validation result
- Retry counter

Process:

- Decide whether to retry
- Trigger new debugging attempt
- Stop if success or max attempts reached

Output:

- Final recovery status

4.3 Recovery Workflow

The recovery process follows this loop:

1. Checkout buggy version
2. Execute test suite
3. Detect failure
4. Localize faulty code
5. Generate repair hypothesis
6. Apply patch
7. Re-run test suite
8. Repeat if necessary

This iterative structure ensures structured and validated recovery.

4.4 Tool Integration

The system integrates external tools for execution and validation:

- Test execution tools (e.g., Maven)
- Code retrieval utilities
- Repository management
- Language model inference engine

Tool usage enhances practical applicability in CI/CD environments.

5. Methodology

5.1 Dataset Description

To evaluate the proposed Failure Recovery System, the **Defects4J** benchmark dataset is used. Defects4J is a widely recognized dataset containing real-world Java bugs collected from open-source projects. Each bug in the dataset includes:

- A buggy version of the source code
- A fixed version of the code
- A set of failing test cases
- A complete test suite

This dataset is suitable for failure recovery research because it provides reproducible bugs along with validation mechanisms through automated testing.

For experimental purposes, a subset of bugs is selected to ensure feasible implementation under limited computational resources.

5.2 Proposed Experimental Setup

The system can be implemented using Python as the orchestration language. The debugging and patch generation components are powered by a reasoning-based Large Language Model (LLM). The implementation environment includes:

- Python programming language
- Maven for running Java test suites
- Quantized LLM model (e.g., CodeLlama-7B or equivalent lightweight model)
- GPU-supported execution environment (such as Google Colab)

For each selected bug:

1. The buggy version is checked out.
2. The test suite is executed to generate failure logs.
3. The multi-agent recovery loop is activated.
4. The process continues until success or maximum retry attempts are reached.

5.3 Model Configuration

The Debugging Agent and Patch Generation Agent utilize a transformer-based LLM configured with:

- Limited token context to handle relevant code sections
- Structured prompts specifying error logs and code snippets
- Maximum retry attempts (e.g., 3 attempts per bug)

Quantized models are used to ensure compatibility with limited GPU resources.

5.4 Evaluation Metrics

To measure system performance, multiple evaluation metrics are defined.

5.4.1 Repair Performance Metrics

- **Repair Success Rate (%)** – Percentage of bugs successfully fixed.
- **Pass@k** – Whether the bug is fixed within k attempts.
- **Patch Correctness** – Whether generated patch matches expected behavior.

5.4.2 Recovery Efficiency Metrics

- **Mean Time to Recovery (MTTR)** – Average time taken to fix a bug.
- **Average Attempts per Bug** – Number of retries required.
- **Execution Cost** – Computational resources used.

5.4.3 Reliability and Safety Metrics

- **Invalid Patch Rate** – Patches that compile but break other tests.
- **Compilation Failure Rate** – Percentage of syntax errors generated.

5.5 Comparative Baseline

To validate improvements, results will be compared against:

- Traditional Automated Program Repair methods (reported literature values)
- Single-step LLM patch generation baseline

This comparison helps evaluate whether the structured multi-agent framework provides measurable benefits.

6. Literature Review

6.1 Classical Automated Program Repair

Early research in Automated Program Repair (APR) focused on search-based and evolutionary approaches. Genetic programming techniques were used to modify code automatically and validate the generated patches using test cases. These systems attempted multiple random or guided code mutations until a test-passing solution was found.

Template-based approaches later introduced predefined repair patterns for common bug types such as null checks or conditional fixes. Probabilistic models were also developed to rank potential patches based on patterns learned from historical fixes.

Although classical APR methods demonstrated that automated repair is possible, they suffer from several limitations:

- Large search space
- Limited reasoning capability
- High computational cost
- Risk of generating patches that pass tests but are semantically incorrect

These limitations motivated the exploration of learning-based techniques.

6.2 Machine Learning-Based Repair

Machine learning approaches introduced statistical and neural models to predict corrections based on learned patterns. Deep learning models were trained on large code datasets to learn mappings between buggy and fixed code snippets.

While these approaches improved patch prediction accuracy, they often focused on small code fragments and lacked repository-scale understanding.

Additionally, they did not incorporate structured validation or iterative repair mechanisms.

As a result, their practical applicability to complex software systems remained limited.

6.3 Large Language Model-Based Repair

With the development of Large Language Models (LLMs), software engineering research shifted toward transformer-based models capable of understanding and generating code. LLMs can analyze error logs, generate explanations, and propose code fixes using natural language prompts.

Benchmarks such as SWE-bench demonstrated that while powerful models show promising reasoning capabilities, their success rates in repository-level repair tasks remain relatively low when used in single-step prompting scenarios. LLMs may produce incomplete, incorrect, or syntactically invalid patches without structured orchestration.

Therefore, LLMs alone are insufficient for reliable automated recovery.

6.4 Agentic AI-Based Recovery Systems

Recent research has introduced tool-augmented and agent-based systems that combine reasoning models with structured workflows. These systems follow iterative loops involving reasoning, tool execution, and validation. Instead of generating a single patch, they repeatedly refine solutions based on feedback.

Agentic frameworks allow AI systems to:

- Read logs
- Search code repositories
- Apply changes
- Execute tests
- Retry if needed

Such approaches have shown improved robustness compared to single-step LLM repair. However, most existing systems still focus primarily on patch generation rather than a complete failure recovery framework integrated with CI/CD-style workflows.

6.5 Comparative Analysis of Existing Approaches

The evolution of repair techniques can be summarized as follows:

Approach	Strength	Limitation
Classical APR	Automated patch search	Limited reasoning
ML-Based Repair	Pattern learning	Limited scalability
LLM-Based Repair	Strong contextual understanding	No structured validation
Agentic Repair	Iterative reasoning + tool usage	Limited recovery-focused metrics

Despite significant advancements, there remains a need for a structured multi-agent system that explicitly targets failure recovery efficiency, incorporates retry strategies, and evaluates recovery time metrics such as MTTR.

6.6 Identified Research Gap

Based on the review, the following gap is identified:

- Existing work focuses mainly on patch generation rather than end-to-end recovery.
- Recovery efficiency metrics are rarely emphasized.
- Structured multi-agent orchestration in CI-style environments remains underexplored.

This motivates the proposed Failure Recovery System using Agentic AI.

7. Conclusion

This report proposes a Failure Recovery System framework design using Agentic AI to automate software debugging and recovery. Traditional debugging and automated repair methods face limitations in reasoning, validation, and recovery efficiency.

The proposed multi-agent architecture integrates failure detection, code analysis, patch generation, and validation within an iterative recovery loop. By combining reasoning-based language models with structured tool execution, the system aims to improve repair success rate and reduce recovery time (MTTR).

The framework demonstrates the potential of agentic AI in enabling more reliable and self-healing software systems for modern development environments.

8.References

Sr. no	Title	Authors(s)	Methods	Model	Description	Data set	Accuracy	Advantages	Disadvantages